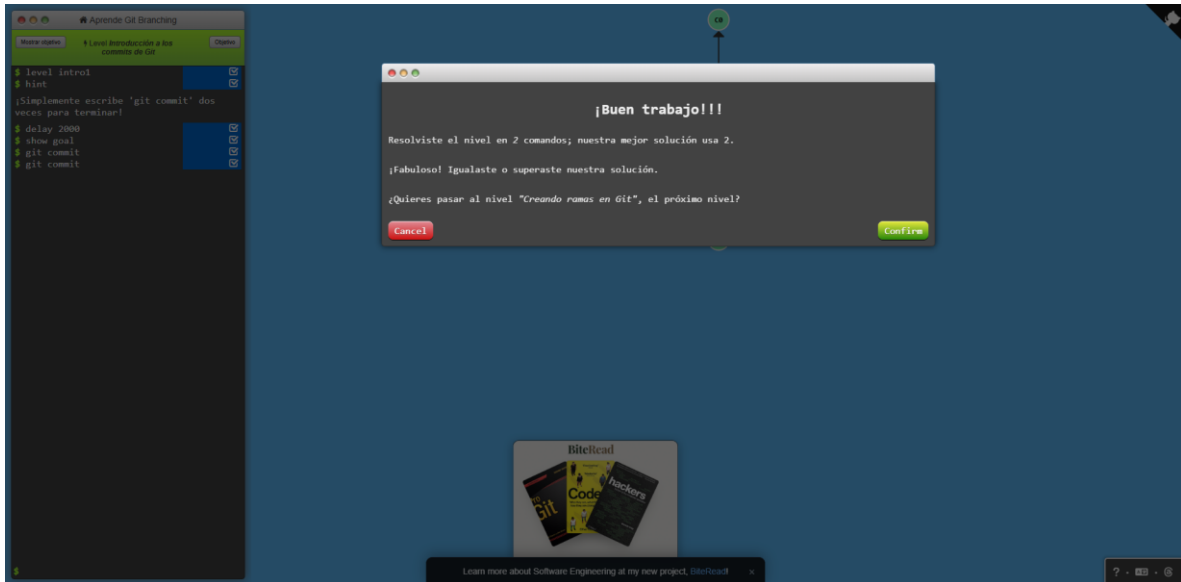


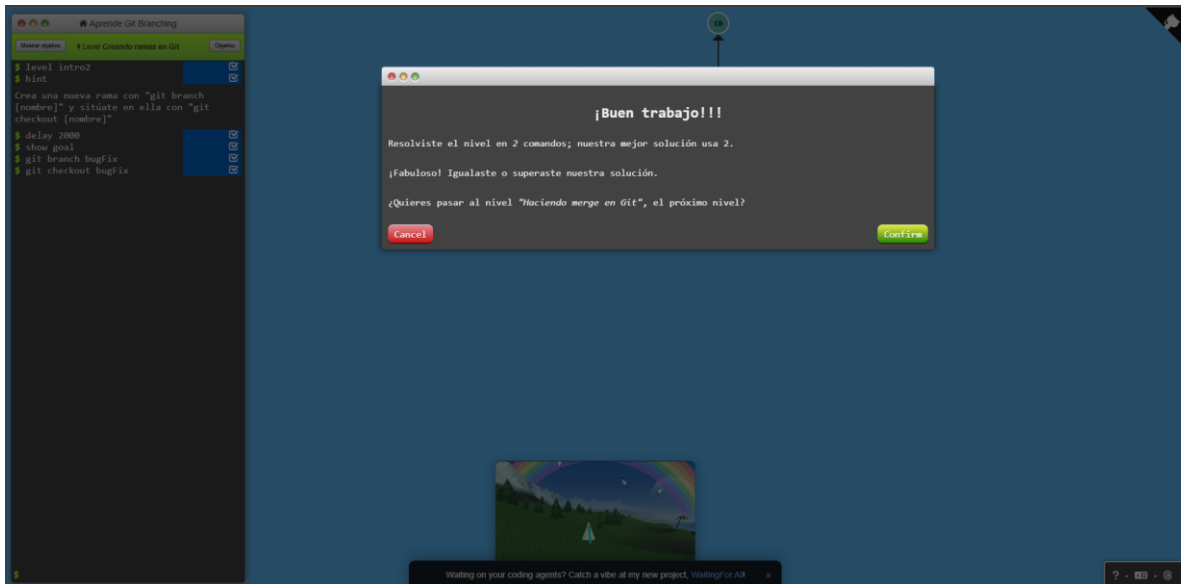
## TALLER GITHUB

### PUNTO1:



**Git Commit:** Crea un nuevo repositorio tomando como base el repositorio padre. Es decir, al generar un nuevo repositorio desde **C2**, se crea el **C3**.

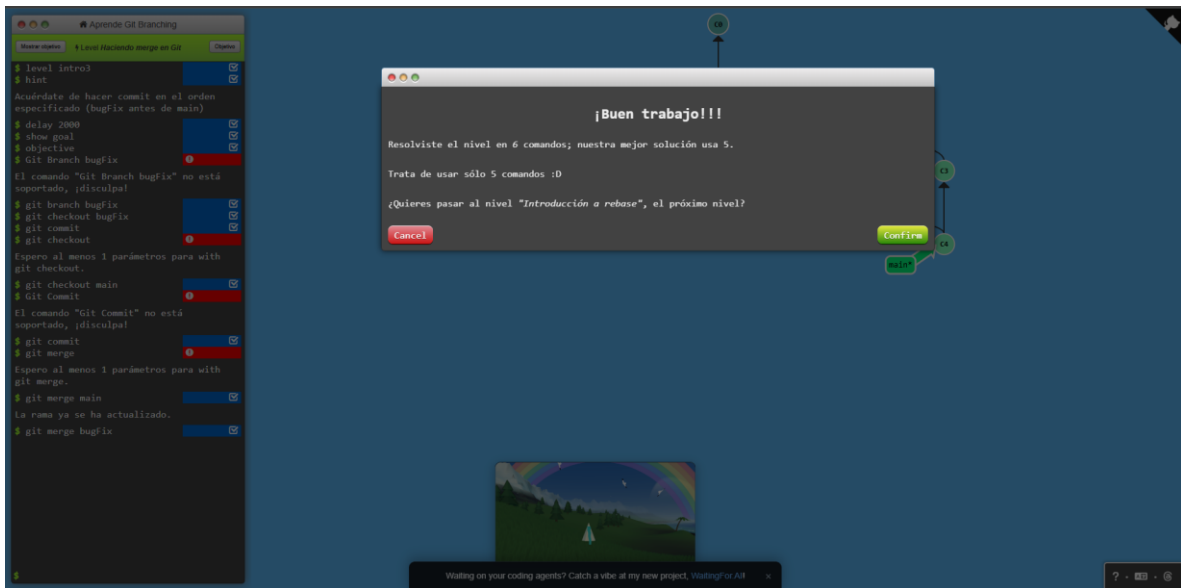
### PUNTO2:



**Git Branch [Nombre]:** Permite crear una rama nueva con un nombre único, la cual hace referencia al commit donde fue creada.

**git checkout [name]:** Nos cambia a esa rama para trabajar en ella antes de realizar un commit con los cambios.

### PUNTO 3:



Git Merge [Nombre]: Crea una rama completamente diferente aparte de la principal para realizar cambios y luego unificarlas para un resultado mejor.

Merge[Nombre]: Lo que significa que cada rama contiene todo el trabajo que hay en el repositorio los une.

- Crea una nueva rama, llamada bugFix
- Haz checkout de la rama bugFix usando git checkout bugFix
- Crea un commit
- Vuelve a main con git checkout
- Crea otro commit
- Haz merge de la rama bugFix a main usando git merge

git branch bugFix

git checkout bugFix

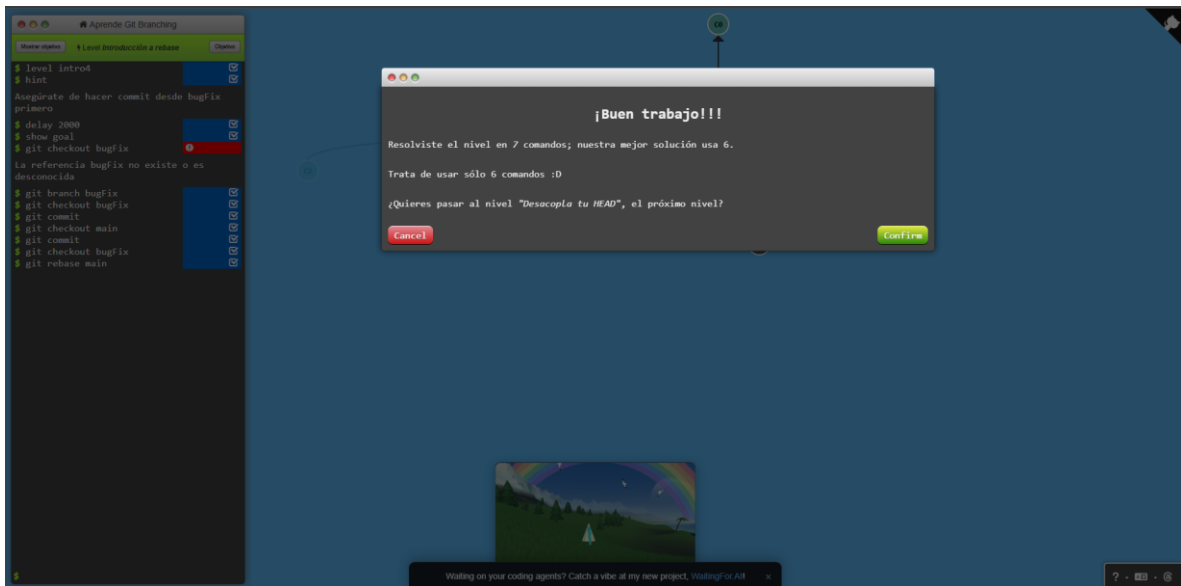
git commit

git checkout

git commit

git merge

#### PUNTO 4:



Git Rebase: Nos gustaría mover nuestro trabajo de bugFix directamente sobre el trabajo de main. De ese modo, parecería que esas dos tareas se desarrollaron secuencialmente, cuando en realidad se hicieron en paralelo.

- Haz checkout de una nueva rama llamada bugFix
- Crea un commit
- Vuelve a la rama main y crea otro commit
- Haz checkout en bugFix otra vez y haz rebase sobre main

git branch bugFix

git checkout bugFix

git commit

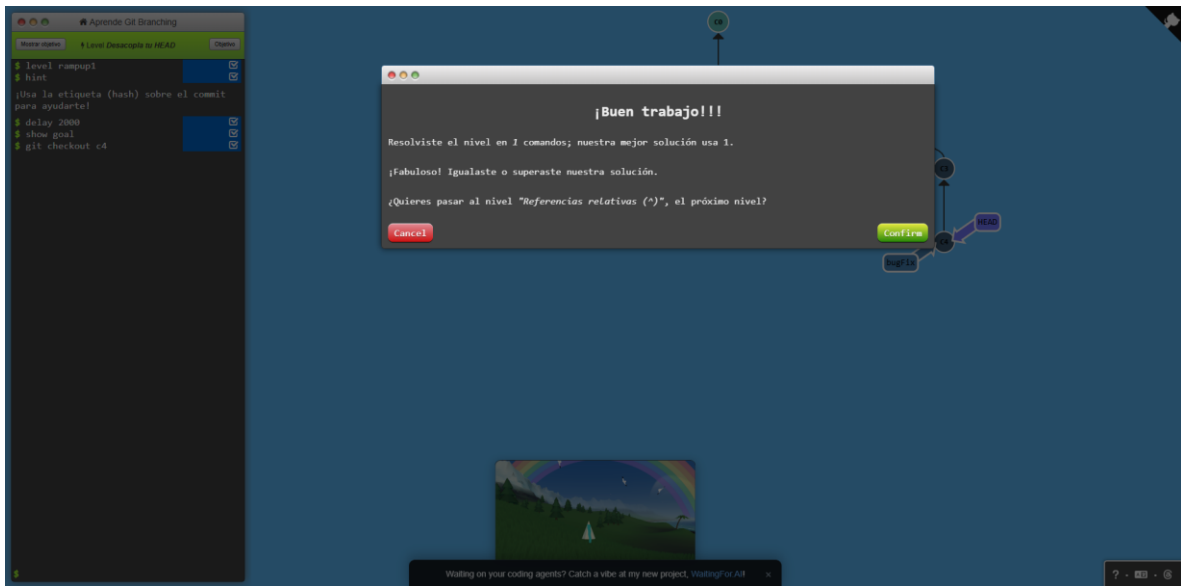
git checkout main

git commit

git checkout bugFix

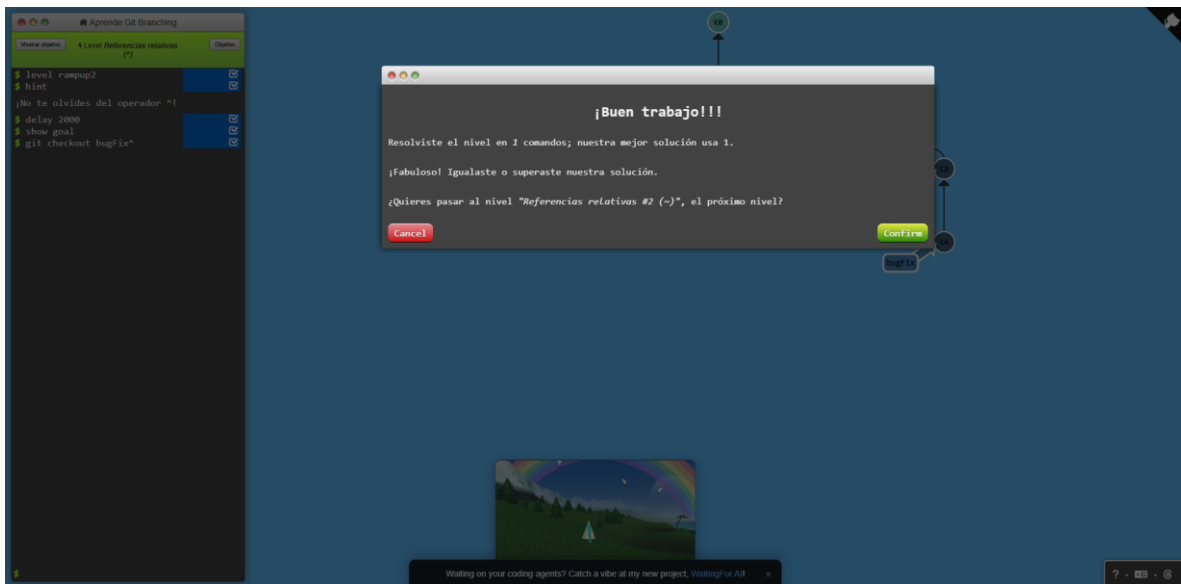
git rebase main

## PUNTO 5:



**Git Checkout C?:** Sirve para mover o posicionar el **HEAD** hacia el commit indicado utilizando la referencia ~.

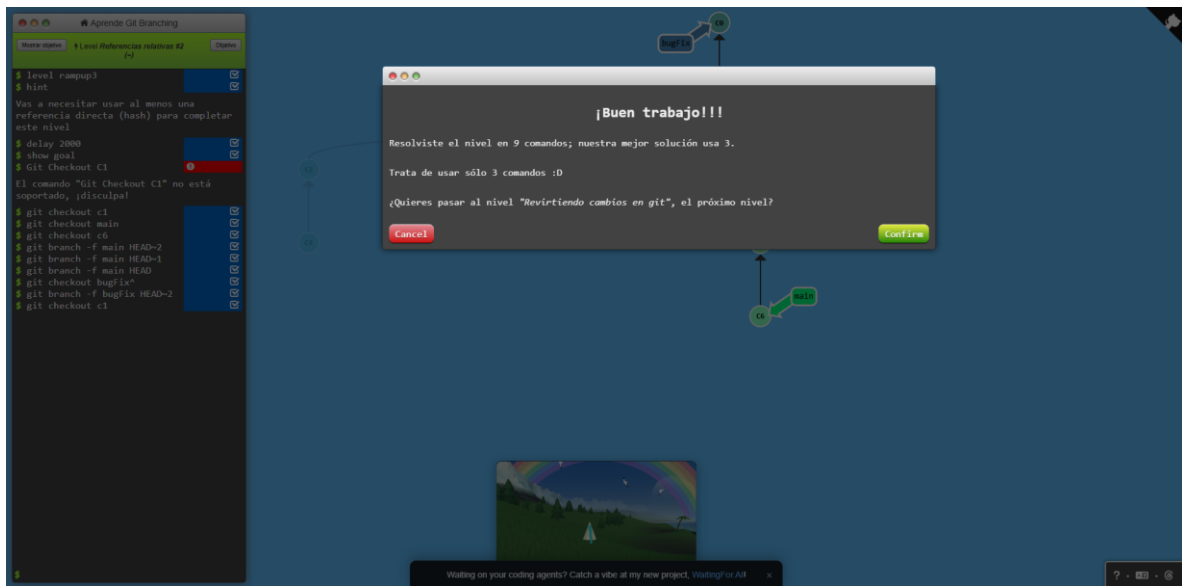
## PUNTO 6:



**Git Checkout [Nombre] ^:** Permite retroceder el **HEAD** un commit por cada símbolo ^ utilizado.

## PUNTO 7:

**git branch -f main HEAD~3:** Mueve la rama **main** tres commits hacia atrás.



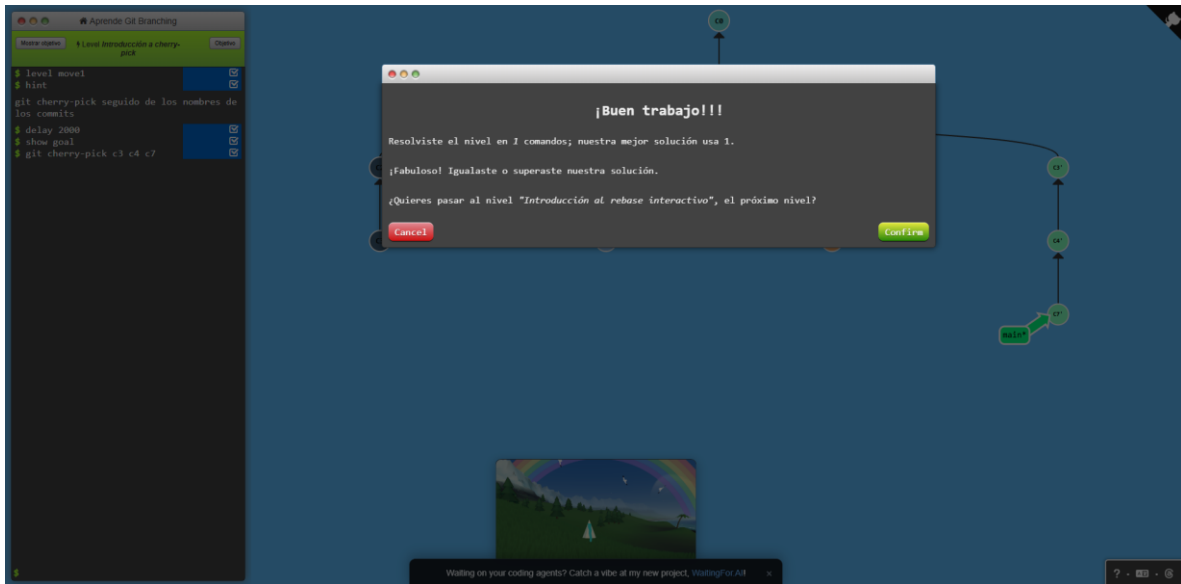
## PUNTO 8:



**Git Revert:** Utiliza un parámetro para indicar el commit que se desea revertir o corregir, generando un nuevo commit de reversión.

**Git Reset:** También utiliza un parámetro para señalar el commit, pero en este caso elimina esos cambios de forma local.

## PUNTO 9:

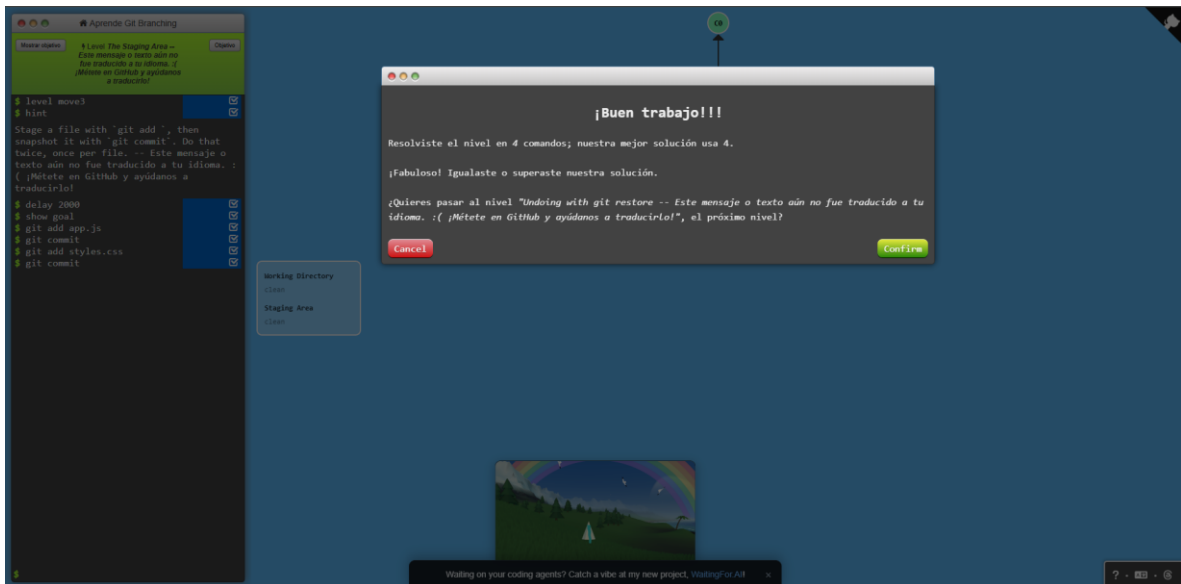


**git cherry-pick:** `git cherry-pick <Commit1> <Commit2> <...>` permite copiar uno o varios commits específicos sobre la ubicación actual (**HEAD**).

PUNTO 10:

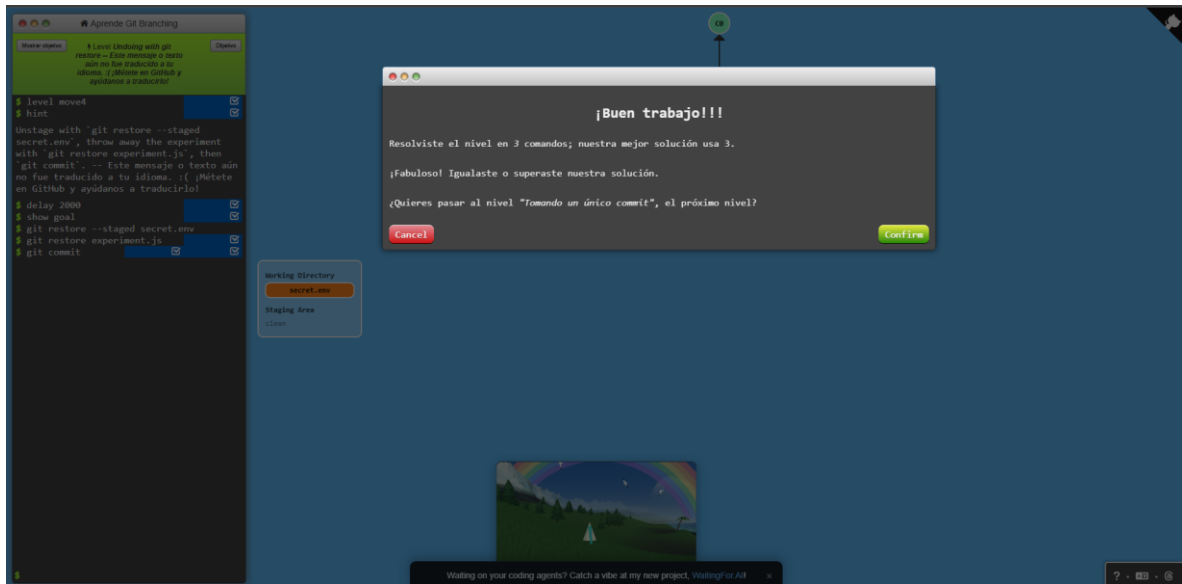
**Git rebase -i HEAD~4:** Permite seleccionar varios commits para reorganizarlos según sea necesario.

PUNTO 11:



**Git Add:** Agrega el archivo indicado al área de preparación (staging) para incluirlo posteriormente en un commit.

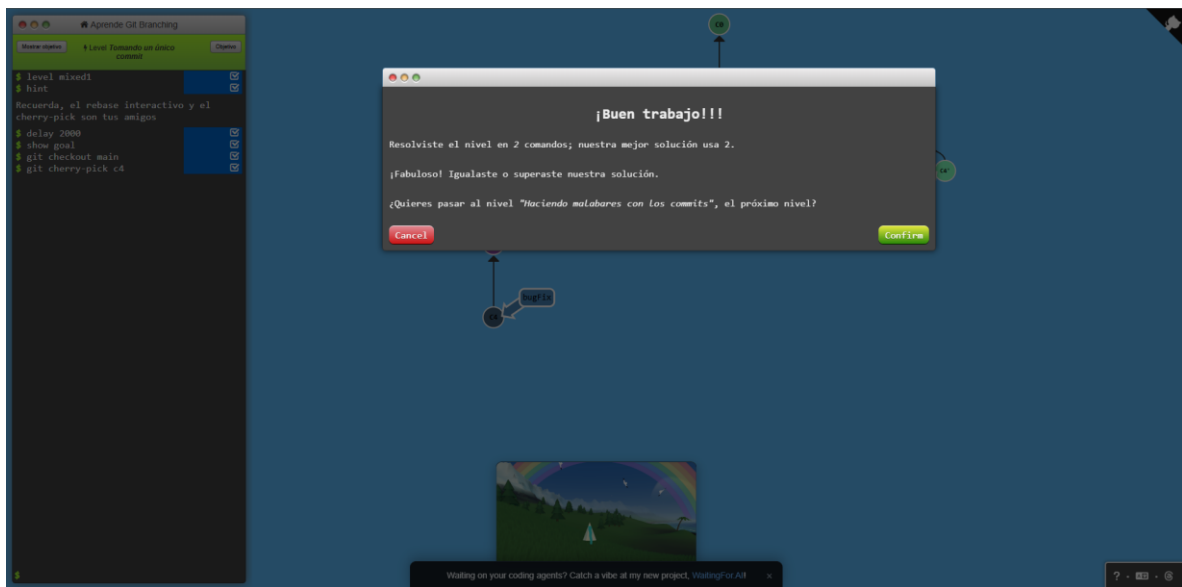
PUNTO 12:



**git restore --staged <file>:** Quita un archivo del área de preparación sin eliminar sus cambios.

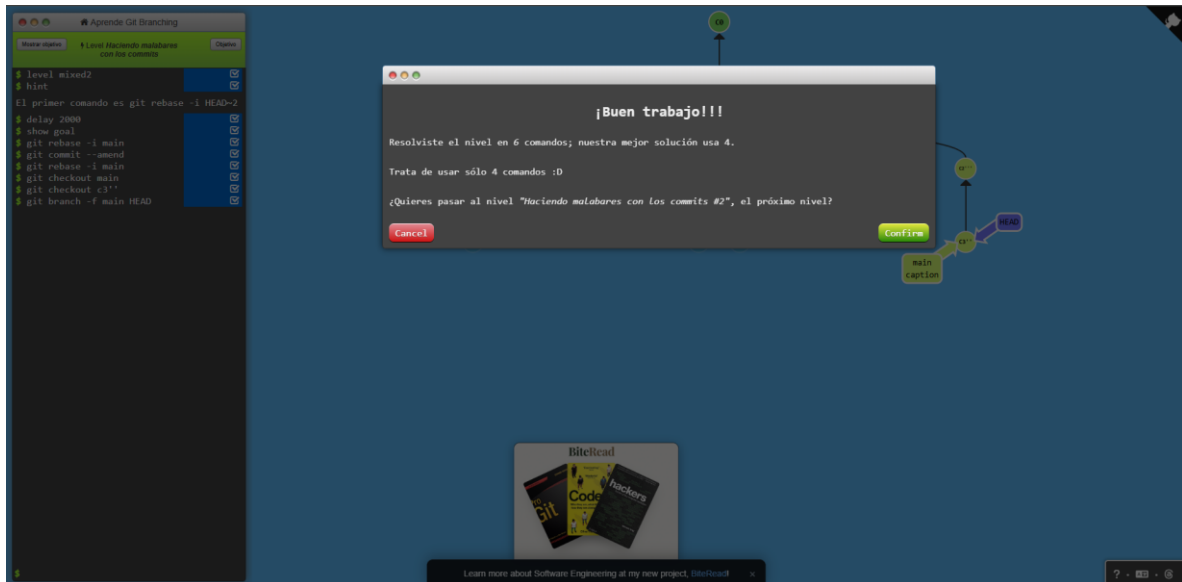
**git restore <file>:** Descarta completamente los cambios realizados en un archivo.

PUNTO 13:



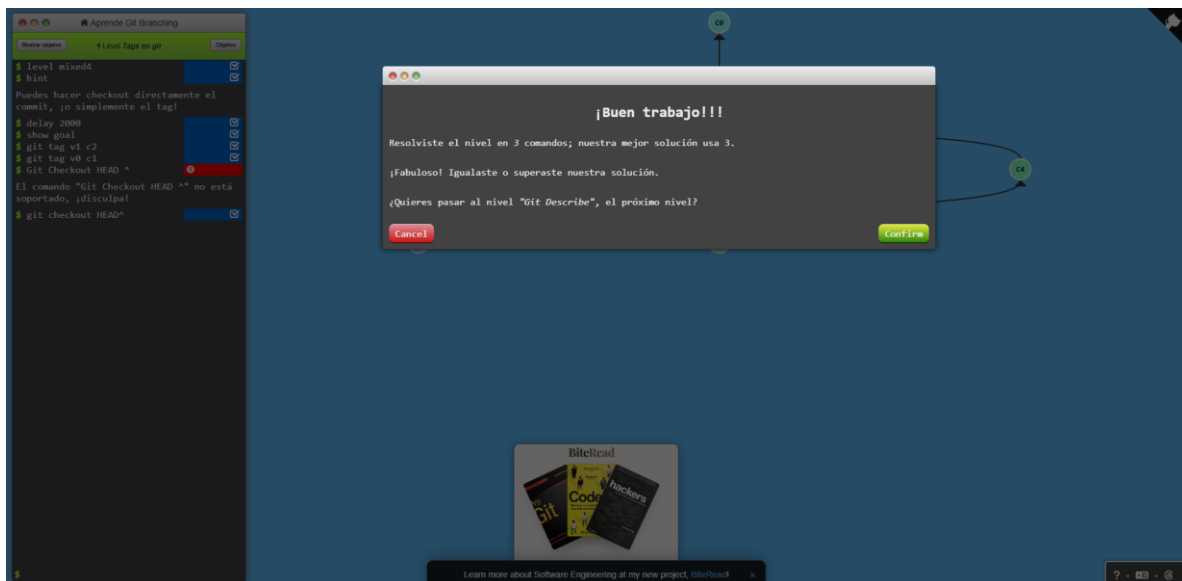
**git rebase -i** y **git cherry-pick**: Permiten reorganizar commits y copiar commits específicos según se requiera.

PUNTO 14:



- Primero se reorganizan los commits con **git rebase -i** para colocar arriba el que se desea modificar.
- Luego se utiliza **git commit --amend** para realizar la modificación.
- Después se vuelven a ordenar los commits con **git rebase -i**.
- Finalmente, se mueve **main** hacia el historial actualizado utilizando el método que se prefiera.

PUNTO 15:

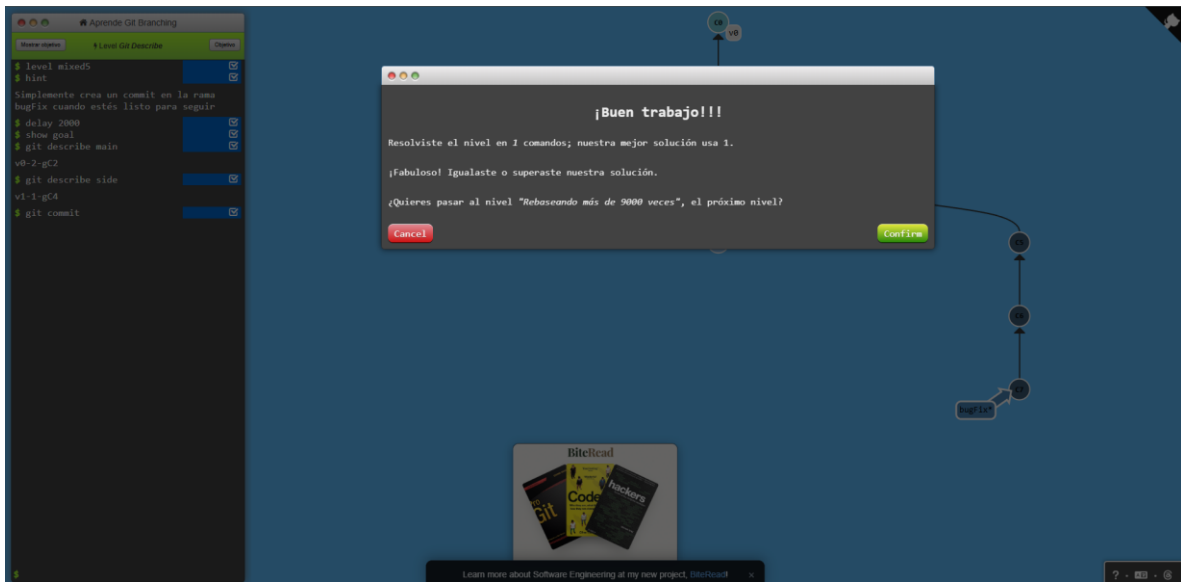


Para este nivel, se crean los **tags** según la visualización final y luego se realiza **checkout** sobre **v1**. Al hacerlo, se entra en estado **detached**, ya que no es posible realizar commits directamente sobre un tag.

**Git tag <nombre> <donde va>:** Crea un tag en el commit que se indique.



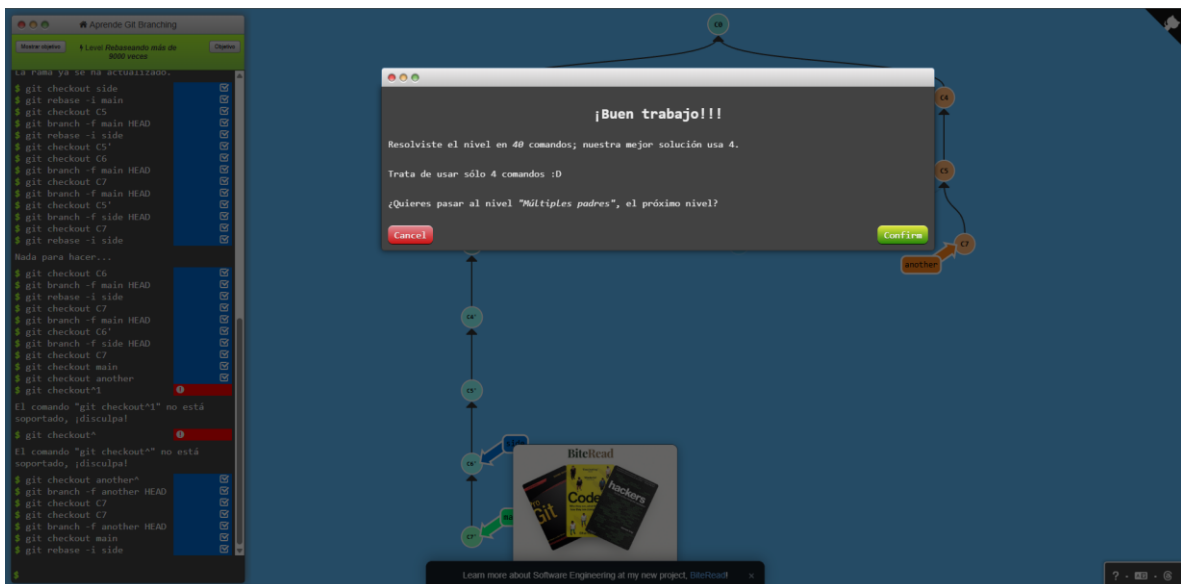
## PUNTO 16:



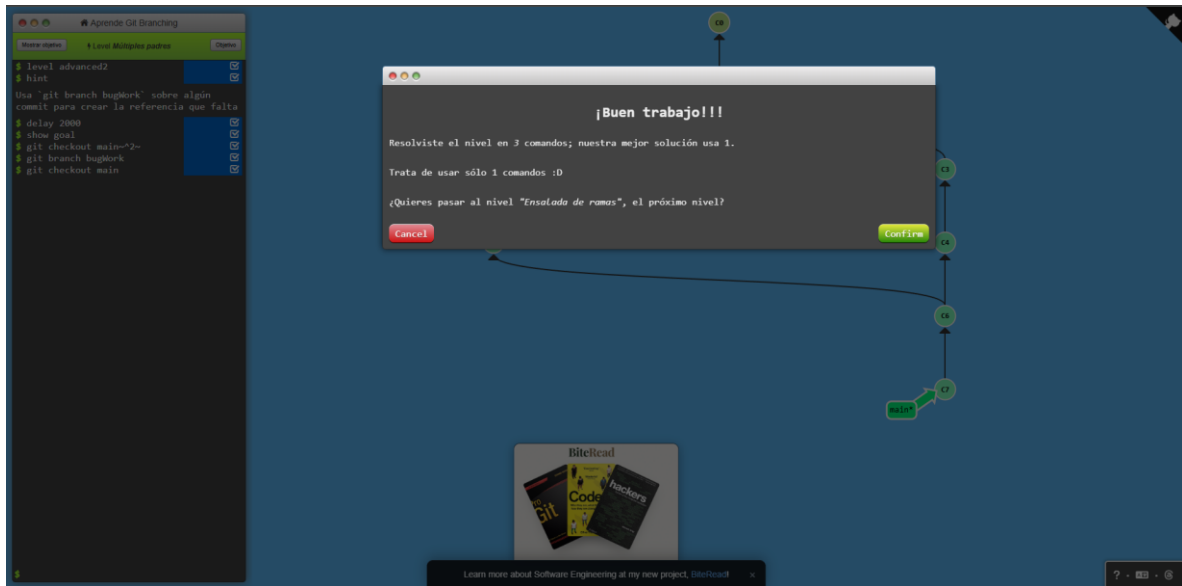
**git describe <ref>:** Describe un commit utilizando una referencia. Si no se especifica ninguna, Git toma el commit actual (HEAD).

**<tag>-<numCommits>-g<hash>:** Indica el tag más cercano, la cantidad de commits de diferencia y el hash del commit descrito.

## PUNTO 17:

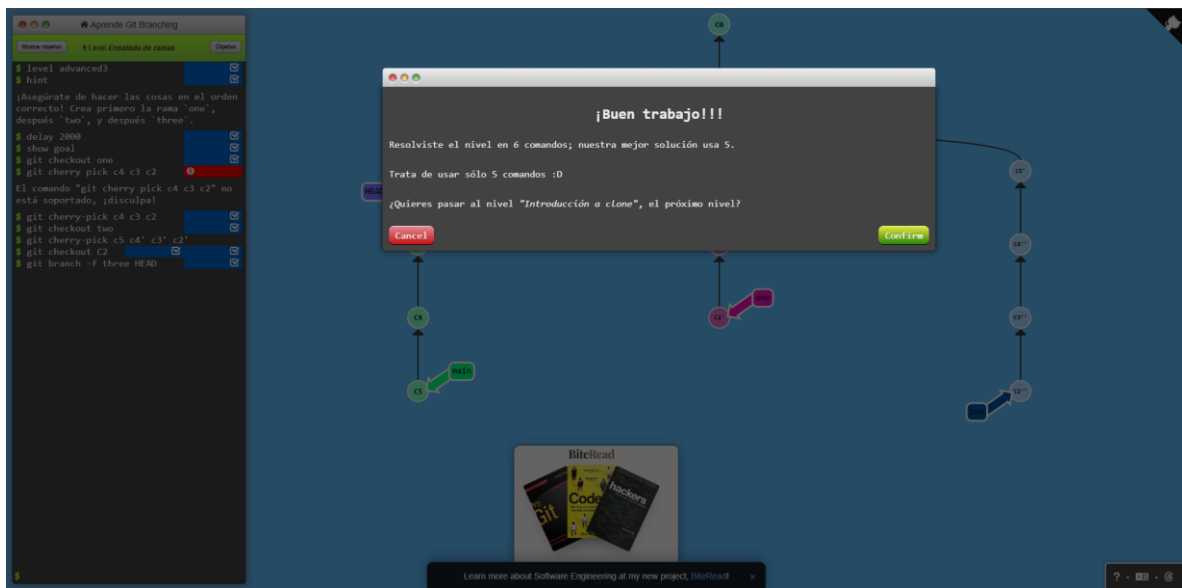


## PUNTO 18:

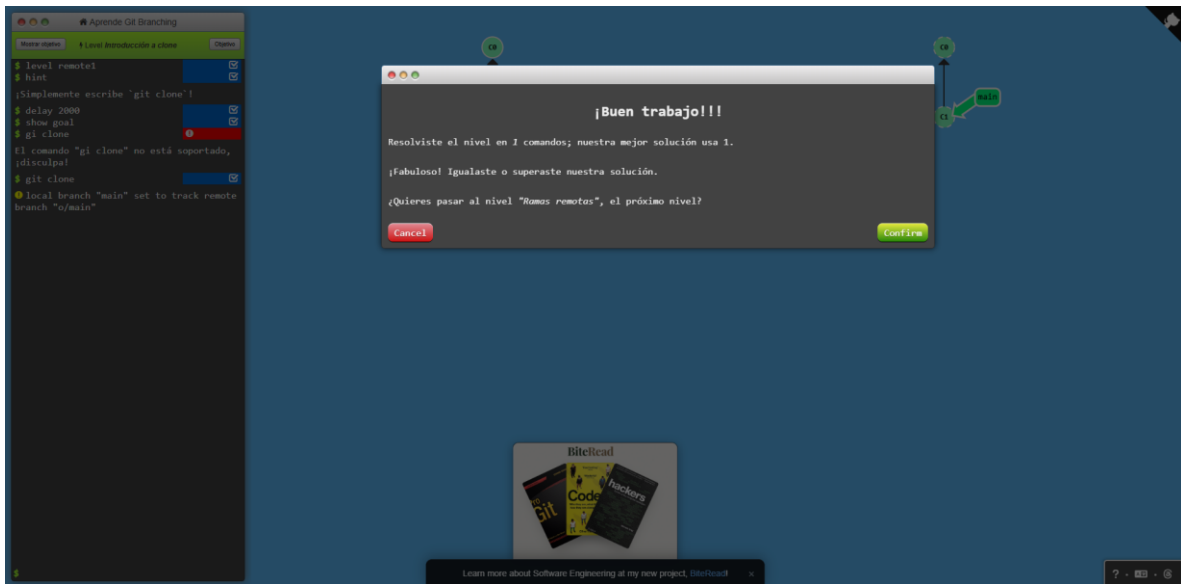


**Git checkout (~) y (^):** El símbolo ^ retrocede un commit por cada uso, mientras que ~ permite regresar varios commits según el número indicado.

PUNTO 19:



## PUNTO 20:



**Git Clone:** Realiza una copia completa del repositorio seleccionado en una ubicación diferente.